INSTITUTE OF ENGINEERING AND MANAGEMENT,KOLKATA

Ride-Sharing Simulation System with Dynamic Dispatch and Fare Computation

Submitted by:
ARPAN GHOSH(A-06)

Department: CSE(IoTCSBT)
Enrolment Number: 12024052017023

Course & Semester: B.Tech. 4th Semester

Academic Year: 2025-2026

Subject Code: PCCCS495

Instructors: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Subject Name: Advanced Programming (OOP) Lab

Institute of Engineering & Management
IEM
श्रद्धाबान लभते ज्ञानम्
Good Education, Good Jobs

2. Problem Statement

Urban commuters frequently face difficulties in finding nearby drivers quickly, obtaining transparent fare estimates, and receiving real-time updates during their journey. Existing local transport systems often lack proper coordination between riders and drivers, leading to inefficient trip management and poor user experience.

The Ride-Sharing Simulation System addresses this problem by modelling the complete lifecycle of a trip through a structured, object-oriented architecture. The system simulates the core operations of a ride-sharing platform — driver and rider registration, automatic driver-rider matching, dynamic fare computation using interchangeable pricing strategies, and real-time status notifications — while ensuring all trip and driver data is persisted to local files across sessions.

This project demonstrates how real-world platform logic can be effectively modelled using core OOP principles and industry-standard design patterns, without relying on external APIs or databases.

3. System Design

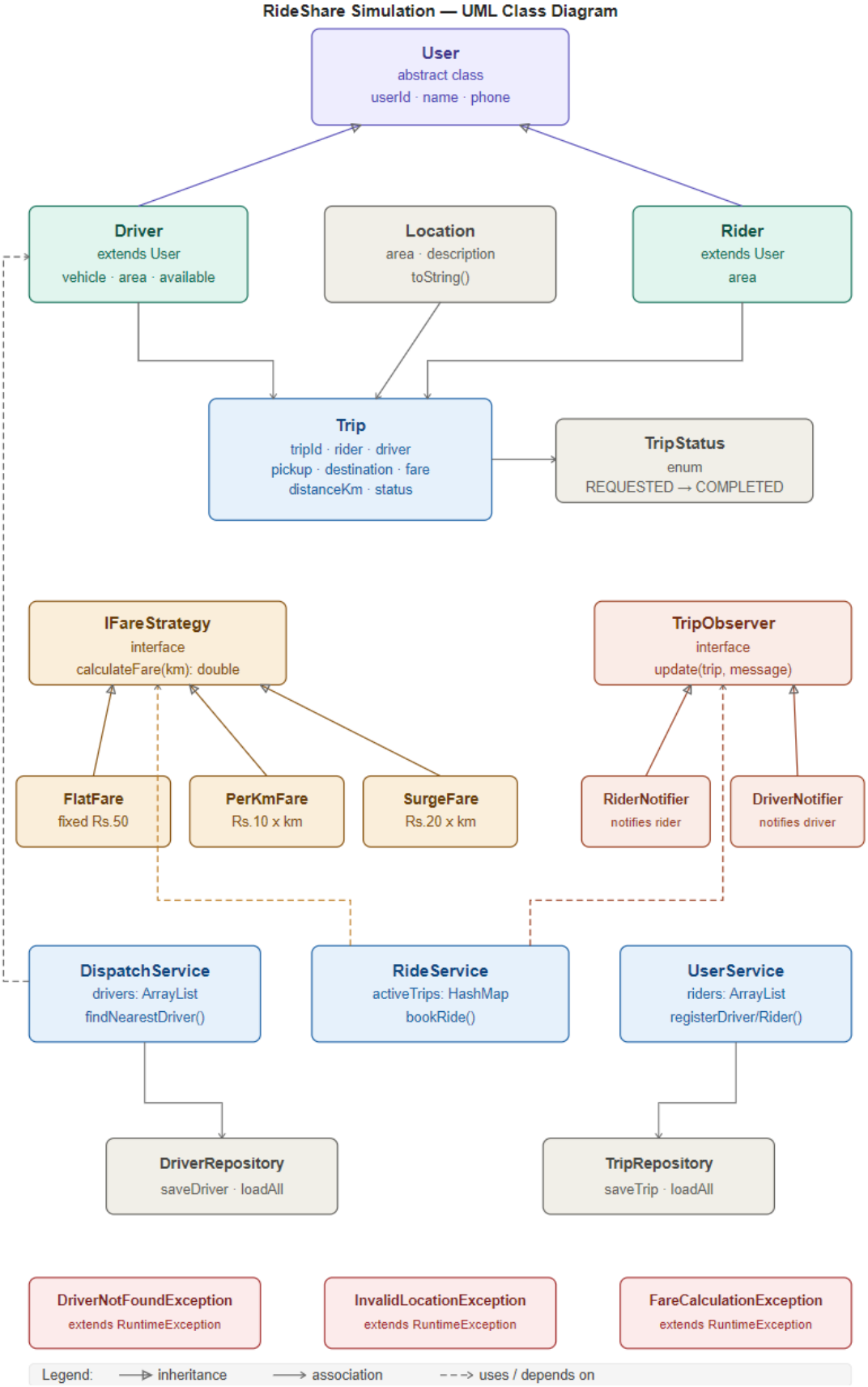
3.1 UML Class Diagram

The UML Class Diagram illustrates the complete architecture of the Ride-Sharing Simulation System, organized across five distinct layers. At the top, the abstract User class forms the base of the inheritance hierarchy, with Driver and Rider extending it to inherit common identity attributes while adding their own domain-specific fields. The Trip class acts as the central entity, connecting a Rider, a Driver, two Location objects, a calculated fare, and a TripStatus enum that tracks the ride lifecycle from REQUESTED through to COMPLETED.

The middle section demonstrates the two core design patterns applied in this project. The Strategy Pattern is implemented through the IFareStrategy interface with three concrete classes — FlatFareStrategy, PerKmFareStrategy, and SurgeFareStrategy — each providing a different pricing algorithm selectable at runtime. The Observer Pattern is implemented through the TripObserver interface, with RiderNotifier and DriverNotifier automatically notifying both parties whenever the trip status changes.

The lower layers handle all coordination and persistence. The service layer contains DispatchService for automatic driver matching, RideService for orchestrating the complete trip lifecycle, and UserService for managing registrations. These services connect to the

Repository layer — DriverRepository and TripRepository — which handle file-based data persistence across sessions. At the bottom, three custom exception classes extend RuntimeException to handle domain-specific failure scenarios gracefully throughout the application.



3.2 Package Structure

The project follows a clean layered package structure under com.rideshare:

```
com.rideshare
model/-User,Driver,Rider,Trip,Location,TripStatus
strategy/-IFareStrategy,FlatFareStrategy,PerKmFareStrategy,SurgFareStrategy
observer/-TripObserver,RiderNotifier,DriverNotifier
dispatcher/-DispatchService
repository/-DriverRepository, TripRepository
service/-RideService, UserService
exception/-DriverNotFoundException,InvalidLocationException,
FareCalculationException
ui/-MainFrame,HomePanel,DriverPanel,RegisterRiderPanel,BookRidePanel,
TripHistoryPanel
```

3.3 Description of Major Classes

Class	Responsibility
User (abstract)	Base class for Driver and Rider. Encapsulates common attributes — userId, name, phone. Declares abstract getRole() method.
Driver	Extends User. Adds vehicleNumber, area, and isAvailable flag. Represents a registered driver in the system.
Rider	Extends User. Adds area attribute. Represents a registered rider who can book trips.
Trip	Core entity connecting Rider, Driver, pickup Location, destination, fare, distance, and TripStatus.
DispatchService	Maintains an ArrayList of available drivers. Finds and assigns the nearest available driver to a ride request.
RideService	Orchestrates the complete trip lifecycle. Uses IFareStrategy for fare calculation and notifies observers on status change.
UserService	Manages Rider and Driver registration. Loads persisted drivers from file on startup.
DriverRepository	Handles file-based persistence for Driver records using BufferedWriter and BufferedReader.
TripRepository	Handles file-based persistence for completed Trip records.

3.4 Interaction Flow

The system follows this sequential interaction flow when a ride is booked:

```
1. User registers a Driver via DriverPanel → UserService → DriverRepository
(saved to file)
2. User registers a Rider via RegisterRiderPanel → UserService
3. User books a ride via BookRidePanel
      a. BookRidePanel calls RideService.bookRide()
      b. RideService calls DispatchService.findNearestDriver()
      c. A Trip object is created with REQUESTED status
      d. IFareStrategy.calculateFare() computes the fare
      e. Trip status updated to ACCEPTED then COMPLETED
      f. TripObserver.update() notifies Rider and Driver
      g. TripRepository saves the completed trip to file
4. User views trip history via TripHistoryPanel →
RideService.getTripHistory()
```

4. OOP Concepts Demonstrated

4.1 Abstraction

Where used: The User abstract class and IFareStrategy interface both demonstrate abstraction.

Why used: User defines shared behaviour (getUserId, getName, getPhone, getRole) without committing to a concrete implementation. IFareStrategy abstracts the fare calculation algorithm, allowing runtime substitution of pricing models without modifying the core booking logic.

```
public abstract class User {
    private String userId;
    private String name;
    private String phone;

    public User(String userId, String name, String phone) {
        this.userId = userId;
        this.name = name;
        this.phone = phone;
    }

    public abstract String getRole(); // subclasses must implement
}
```

The abstract getRole() method forces every subclass (Driver, Rider) to declare its own role. The caller never needs to know the concrete type — only that it is a User.

4.2 Inheritance

Where used: Driver and Rider both extend the abstract User class.

Why used: Both entities share common identity attributes (userId, name, phone) but have different domain-specific fields. Inheritance eliminates code duplication while allowing specialisation.

```
public class Driver extends User {
    private String vehicleNumber;
    private String area;
    private boolean isAvailable;

    public Driver(String userId, String name, String phone,
                  String vehicleNumber, String area) {
        super(userId, name, phone); // calls User constructor
        this.vehicleNumber = vehicleNumber;
        this.area = area;
        this.isAvailable = true;
    }

    @Override
    public String getRole() { return "DRIVER"; }
}
```

The super() call delegates identity initialisation to the User class. The @Override annotation confirms that getRole() satisfies the abstract contract defined in User.

4.3 Polymorphism

Where used: The Strategy Pattern implementation in the fare calculation layer demonstrates runtime polymorphism.

Why used: Three concrete fare strategies — FlatFareStrategy, PerKmFareStrategy, and SurgeFareStrategy — each implement the IFareStrategy interface differently. The RideService holds a reference to IFareStrategy and invokes calculateFare() without knowing the concrete type at compile time.

```
public interface IFareStrategy {
    double calculateFare(double distanceKm);
    String getStrategyName();
}

public class SurgeFareStrategy implements IFareStrategy {
    private static final double RATE = 10.0;
    private static final double MULTIPLIER = 2.0;

    @Override
```

```
    public double calculateFare(double distanceKm) {
        return RATE * distanceKm * MULTIPLIER;
    }
}
```

At runtime, the user selects a fare type in the UI. The correct strategy object is passed to bookRide(). The RideService calls fareStrategy.calculateFare(distance) — the same method call produces different results depending on which strategy is active. This is classic runtime polymorphism.

4.4 Encapsulation

Where used: All model classes — User, Driver, Rider, Trip, Location — declare their fields as private and expose only controlled access through public getters and setters.

Why used: Encapsulation protects the internal state of objects from direct external modification, ensuring data integrity throughout the application.

```
public class Trip {
    private String tripId;
    private Rider rider;
    private Driver driver;
    private double fare;
    private TripStatus status;

    public void setFare(double fare) { this.fare = fare; }
    public void setStatus(TripStatus status) { this.status = status;
}
    public double getFare() { return fare; }
    public TripStatus getStatus() { return status; }
}
```

The fare and status fields can only be changed through the defined setter methods. This ensures that no external class can arbitrarily modify a Trip's state — all changes are controlled and intentional.

4.5 Design Pattern — Strategy Pattern

Where used: Fare calculation in RideService.

Why used: The system supports three different pricing models. Rather than using conditional logic (if-else or switch), the Strategy Pattern allows the pricing algorithm to be selected and

swapped at runtime without changing the RideService code. This follows the Open-Closed Principle — open for extension, closed for modification.

```
// In RideService.bookRide():
double fare = fareStrategy.calculateFare(distanceKm);
trip.setFare(fare);

// In BookRidePanel, strategy selected by user:
IFareStrategy strategy;
if (selectedFare == 0) strategy = new FlatFareStrategy();
else if (selectedFare == 1) strategy = new PerKmFareStrategy();
else strategy = new SurgeFareStrategy();
```

The RideService is completely unaware of which strategy is being used. It simply calls calculateFare() on the interface reference. Adding a new pricing model in the future requires only creating a new class that implements IFareStrategy — no existing code needs to change.

4.6 Design Pattern — Observer Pattern

Where used: Trip status notifications in RideService.

Why used: When a trip status changes, multiple parties (Rider and Driver) need to be notified. The Observer Pattern decouples the notification logic from the core booking logic, making the system extensible — new notification channels can be added without modifying RideService.

```
public interface TripObserver {
    void update(Trip trip, String message);
}

// In RideService:
private void notifyObservers(Trip trip, String message) {
    for (TripObserver observer : observers) {
        observer.update(trip, message);
    }
}

// Called after driver assignment:
notifyObservers(trip, "Driver " + driver.getName() + " is on the
way!");
```

RiderNotifier and DriverNotifier both implement TripObserver. When notifyObservers() is called, each registered observer receives the trip and message. The RideService does not

reference RiderNotifier or DriverNotifier directly — it only knows TripObserver. This is loose coupling in practice.

5. Implementation Highlights

5.1 Automatic Driver Matching

The DispatchService maintains an ArrayList of registered and available drivers. When a ride is requested, it iterates through the list and returns the first available driver. If no driver is available, a DriverNotFoundException is thrown, which is caught in the UI layer and displayed as a user-friendly message.

```
public Driver findNearestDriver(String riderArea) {
    if (availableDrivers.isEmpty()) {
        throw new DriverNotFoundException(
            "No drivers registered yet.");
    }
    for (Driver driver : availableDrivers) {
        if (driver.isAvailable()) {
            return driver;
        }
    }
    throw new DriverNotFoundException(
        "No driver available at the moment.");
}
```

This method demonstrates both the use of Java Collections (ArrayList) and custom exception handling in a single, focused method.

5.2 File-Based Persistence

The DriverRepository class saves driver records to a plain text file (data/drivers.txt) using comma-separated values. On application startup, UserService loads all previously saved drivers from this file, making them immediately available for ride assignments.

```
public void saveDriver(Driver driver) {
    try (BufferedWriter writer = new BufferedWriter(
            new FileWriter(FILE_PATH, true))) {
        writer.write(driver.getUserId() + "," +
                     driver.getName() + "," +
                     driver.getPhone() + "," +
                     driver.getVehicleNumber() + "," +
                     driver.getArea());
```

```
            writer.newLine();
        } catch (IOException e) {
            System.out.println("Error saving driver: " + e.getMessage());
        }
    }
```

The try-with-resources construct ensures the BufferedWriter is always closed after writing, preventing resource leaks. The append flag (true) ensures new drivers are added to the file without overwriting existing records.

5.3 Complete Trip Lifecycle in RideService

The bookRide() method in RideService coordinates the entire trip from request to completion in a single transactional flow, demonstrating how multiple OOP concepts work together.

```
public Trip bookRide(Rider rider, Location pickup,
                     Location destination, double distanceKm,
                     IFareStrategy fareStrategy) {
    Driver driver =
dispatchService.findNearestDriver(rider.getArea());
    driver.setAvailable(false);
    String tripId = "TRIP" + tripCounter++;
    Trip trip = new Trip(tripId, rider, driver, pickup, destination,
distanceKm);
    double fare = fareStrategy.calculateFare(distanceKm);
    trip.setFare(fare);
    trip.setStatus(TripStatus.ACCEPTED);
    activeTrips.put(tripId, trip);
    notifyObservers(trip, "Driver " + driver.getName() + " is on the
way!");
    trip.setStatus(TripStatus.COMPLETED);
    driver.setAvailable(true);
    tripHistory.add(trip);
    return trip;
}
```

This single method demonstrates abstraction (IFareStrategy), collections (HashMap activeTrips, ArrayList tripHistory), observer pattern (notifyObservers), and encapsulation (trip.setStatus, driver.setAvailable) all working together.

This is the most architecturally significant method in the entire project. It serves as the point where every layer of the system converges — the model layer provides the Trip and Driver objects, the strategy layer computes the fare, the observer layer delivers notifications, and the repository layer persists the result. The fact that this method remains clean and readable despite coordinating so many components is a direct result of the separation of concerns enforced by the layered architecture. Each collaborating class does exactly one job, and RideService simply orchestrates them.

6. Testing and Error Handling

6.1 Test Cases Considered

Test Case	Expected Behaviour
Register driver then book ride	Driver assigned successfully, fare calculated, trip saved
Book ride with no driver registered	DriverNotFoundException thrown, UI shows error message
Enter non-numeric distance	NumberFormatException caught, UI prompts valid input
Leave fields empty and book	Validation check triggers, user prompted to fill all fields
Register multiple drivers, book ride	First available driver assigned correctly
Book for Me with no registered rider	UI shows message to register first
Reopen app after registering driver	Driver loaded from drivers.txt, available for booking
Select Surge fare for 10km trip	Fare = Rs.200 (10 x 10 x 2)

6.2 Custom Exception Classes

Three custom exception classes extend RuntimeException to handle domain-specific failure scenarios:

- DriverNotFoundException: Thrown when no available driver exists in the system.
- InvalidLocationException: Thrown when pickup or destination location is null or empty.
- FareCalculationException: Thrown when fare computation encounters invalid input.

All exceptions are caught at the UI layer (BookRidePanel) and displayed as readable messages in the output area, ensuring the application never crashes on invalid user input.

6.3 Edge Cases Handled

- Empty form fields: Checked before any service call is made.
- Duplicate driver loading: DriverRepository checks file integrity on load.
- Driver availability: Driver is marked unavailable during a trip and restored to available on completion.
- Negative or zero distance: Caught by NumberFormatException handling in BookRidePanel.

7. Git Workflow

The project was developed incrementally over multiple days with meaningful, descriptive commit messages reflecting the progression of development. Each commit represents a focused, logical unit of work.

Day	Commit Message
Day 1	set up complete package structure for the ride sharing project
Day 2	added User as abstract base class along with Driver Rider and Location models
Day 3	added Trip TripStatus models along with custom exception classes
Day 4	implemented Strategy Pattern with IFareStrategy and all fare strategy classes
Day 5	implemented Observer Pattern with TripObserver RiderNotifier and DriverNotifier
Day 6	added DispatchService for driver matching and Repository layer for file persistence
Day 7	added RideService and UserService to handle core business logic
Day 8	added complete Swing UI with all panels and navigation
Day 9	added UML class diagram to docs folder
Day 10	final project cleanup and documentation update

Each commit was pushed to the GitHub Classroom repository individually, demonstrating genuine incremental development rather than a single bulk upload. Screenshots of the full commit history are available in the submitted repository.

8. Conclusion and Future Scope

8.1 Conclusion

The Ride-Sharing Simulation System successfully demonstrates a well-structured, object-oriented Java application. The project applies core OOP principles — abstraction, inheritance, polymorphism, and encapsulation — alongside two industry-standard design patterns: the Strategy Pattern for fare calculation and the Observer Pattern for real-time notifications.

The layered architecture ensures clear separation of concerns between the model, service, repository, and UI layers. File-based persistence allows data to survive across application sessions. The Swing GUI provides a professional, interactive interface that makes the simulation intuitive to demonstrate and defend.

Every design decision in this project was made with extensibility in mind. Adding a new fare strategy requires only implementing IFareStrategy. Adding a new notification channel requires only implementing TripObserver. The core booking logic in RideService does not need to change in either case.

8.2 Future Scope

- Integration with a real maps API for automatic distance and location calculation.
- Database-backed persistence using JDBC or an ORM framework instead of flat files.
- Multi-threading support to simulate concurrent ride requests from multiple riders.
- Driver rating system after trip completion.
- Real-time push notifications using Java messaging libraries.
- Web-based frontend using Spring Boot replacing the Swing GUI.

9.Appendix

9.1 Initial Project Proposal

Project Proposal — Object Oriented Programming Laboratory

[Name: Arpan Ghosh **Roll No:** 06 **Enrollment No:** 12024052017023 **Dept:** CSE (IoTCSBT)]

Ride-Sharing Simulation System with Dynamic Dispatch and Fare Computation

Problem Statement:

Urban commuters frequently struggle to find nearby drivers quickly, get transparent fare estimates, and receive real-time updates during their journey. Existing local transport systems lack proper coordination between riders and drivers. The proposed Ride-Sharing Simulation System models the complete lifecycle of a trip — from ride request to completion — using a structured object-oriented architecture with dynamic fare computation and real-time notifications, demonstrating abstraction, modularity, and design pattern application.

Target User:

Daily commuters, office goers, and independent drivers in urban and semi-urban areas who need a fast, transparent, and organized ride-booking experience.

Core Features:

- Rider and Driver registration with profile management
- Automatic driver-rider matching via nearest-available dispatch algorithm
- Dynamic fare calculation — Flat, Per-Km, and Surge pricing strategies
- Real-time trip status notifications to both Rider and Driver
- Trip history with persistent file-based storage and exception handling

OOP Concepts to be Used:

- **Abstraction:** Abstract User class; FareStrategy interface
- **Inheritance:** Driver and Rider extend User
- **Polymorphism:** Runtime fare strategy switching across FlatFare, PerKmFare, SurgeFare
- **Encapsulation:** Private fields with getters/setters across all entities
- **Design Patterns:** Strategy Pattern (fare engine) + Observer Pattern (notifications)
- **Collections & File I/O:** ArrayList, Queue for data management; file-based persistence
- **Exception Handling:** Custom exceptions for invalid inputs, unavailable drivers, fare errors

Architecture:

Layered OOP architecture: Abstract User class extended by Driver and Rider. RideService manages the full trip lifecycle. FareStrategy enables runtime pricing model selection. Observer Pattern handles decoupled notifications. Repository layer manages all file-based persistence.

User (Abstract) → Driver/Rider → RideService → DispatchService → FareStrategy →TripObserver → Repository

9.2 Approval Email

Outlook

PCCCS495 Proposal Review – APPROVED_MINOR

From soumadip.biswas@ieminternational.ai <soumadip.biswas@ieminternational.ai>
Date Tue 3/10/2026 9:21 AM
To Arpan Ghosh <Arpan.Ghosh2024@iem.edu.in>

Dear Arpan Ghosh,

Project: Ride-Sharing Simulation System with Dynamic Dispatch and Fare Computation
Decision: APPROVED_MINOR
Score: 92

Strengths:
The proposal presents a clear, well-defined problem and a robust, object-oriented solution. It demonstrates a strong understanding of core OOP principles such as Abstraction (abstract User class), Inheritance (Driver, Rider extend User), Polymorphism (dynamic fare computation via Strategy Pattern), and Encapsulation (Trip object, layered architecture). The explicit and correct application of design patterns, specifically the Strategy Pattern for fare calculation and the Observer Pattern for real-time notifications, is a significant strength, indicating a deeper grasp of OOP design principles beyond basic syntax. The architecture is well-structured with clear separation of concerns (DispatchService, Repository Layer). The inclusion of persistence (file-based storage) and exception handling adds realism and robustness to the simulation.

Improvements:
While strong, the proposal could benefit from minor elaborations. Specifying the exact types of Java Collections (e.g., ArrayList for users, HashMap for active trips) intended for use would add clarity. For the Strategy Pattern, explicitly defining an `IFareStrategy` interface with concrete implementations like `FlatFareStrategy`, `PerKmFareStrategy`, and `SurgeFareStrategy` would make the design even more explicit. Additionally, a brief mention of a testing strategy (e.g., unit tests for individual components) would enhance the proposal's completeness.

Regards,
Course Instructor